MODULE 15

Business Logic and Service Layer Design

Building the core of your application — organize business logic into a clean, testable, maintainable service layer.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 15 OVERVIEW

Building the core of your application — the Service Layer contains business logic, orchestrates operations, and manages transactions.

- The Service Layer sits between controllers and the data access layer — it enforces rules, manages transactions, and acts as the bridge between presentation and persistence.

DURATION & LAB

1 Hour Theory

Service/Repository patterns, DI, transactions.

45 Minutes Lab

Service Layer Design.

Scenario

Enforce customer status-transition rules for Northstar CRM.

SERVICE LAYER DOES

Business rules, orchestration, transactions, exceptions

SITS BETWEEN

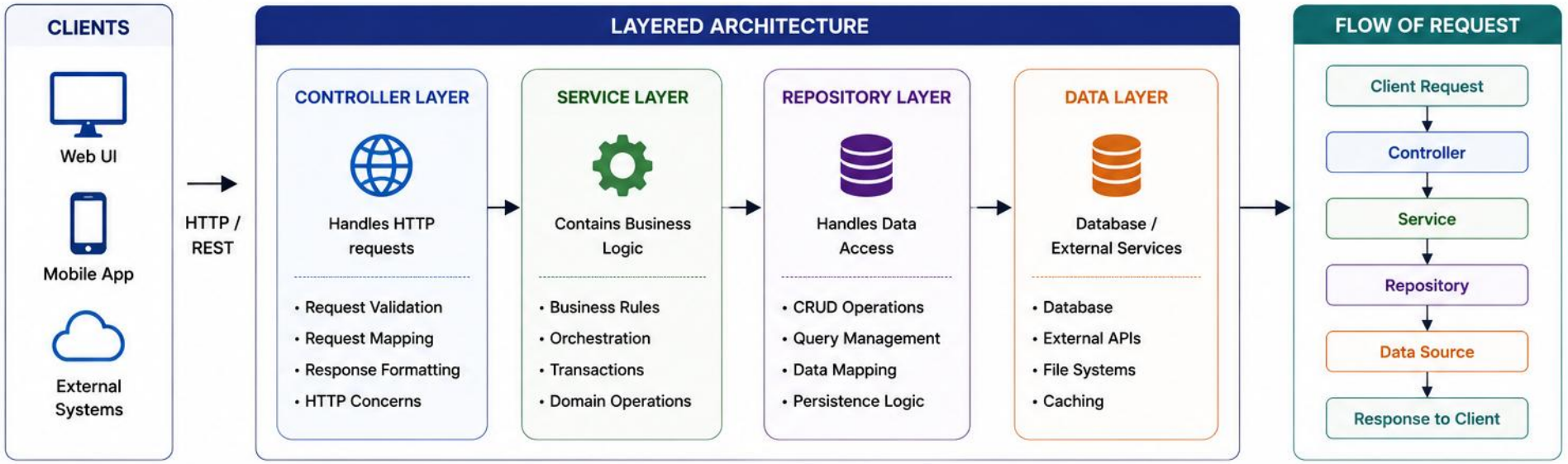
Controllers (presentation) and Repositories (data access)

THE PAYOFF

Clean separation, testability, reusability, scalability

KEY TAKEAWAY A well-designed service layer keeps your business logic organized, reusable, and independent of web and data-access concerns.

SERVICE LAYERS MATTER



WHY SERVICE LAYERS MATTER

SEPARATION OF CONCERNS
Each layer has a specific responsibility.

REUSABILITY
Business logic can be reused across controllers.

TESTABILITY
Layers can be tested in isolation.

MAINTAINABILITY
Changes in one layer have minimal impact on others.

SCALABILITY
Easier to scale and evolve the application.

SECURITY
Centralized security and validation at appropriate layers.

SERVICE LAYER RESPONSIBILITIES

BUSINESS LOGIC
Implements core business rules and processes.

TRANSACTION MANAGEMENT
Manages transactions and ensures data consistency.

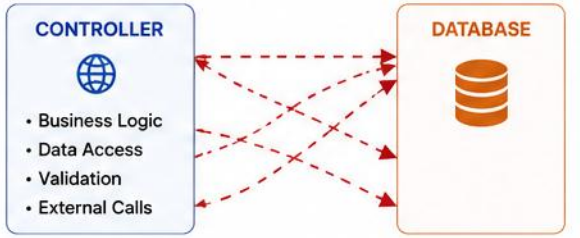
ORCHESTRATION
Coordinates multiple operations and services.

VALIDATION
Performs business validations and checks.

INTEGRATION
Integrates with external systems and services.

AUDITING & LOGGING
Handles audit trails and business logs.

WITHOUT SERVICE LAYER



- Tight coupling between controller and data access
- Duplicate business logic across controllers
- Hard to test and maintain
- Difficult to change or extend
- Violates separation of concerns

SEPARATION OF CONCERNS

Each layer has one job — controllers handle HTTP, services handle business logic, repositories handle data access.

Presentation Layer Controllers handle HTTP requests and responses.

Service Layer Business logic, orchestration, transactions.

Repository Layer Data access — CRUD operations via JPA/DAO.

Database Persistent storage.

WITHOUT: logic scattered across controllers, duplicated, hard to unit test.

Tight coupling between web layer and data access.

WITH: logic centralized, reusable, and easy to test independently.

Clean separation enables independent evolution and reuse.

Why it matters: better testability, readability, flexibility, and team collaboration — each layer evolves and is tested independently.

KEY TAKEAWAY Keep dependencies deliberate and directed toward stable abstractions. Presentation should not access persistence directly, and business logic should remain independent of delivery and storage details.

SEPARATION OF CONCERNS

LAYERED ARCHITECTURE (SEPARATED CONCERNS)

Client

Request

Response

PRESENTATION LAYER
Handles UI, input validation, and request formatting

Concern: User Interface & Presentation

BUSINESS LAYER
Contains business rules, workflows, and logic

Concern: Business Logic & Rules

DATA ACCESS LAYER
Handles data operations, queries, and transactions

Concern: Data Access & Persistence

DATA LAYER
Manages databases, files, and external data sources

Concern: Data Storage & Retrieval

VS

MONOLITHIC ARCHITECTURE (NO SEPARATION)

SINGLE COMPONENT
UI, Business Logic, Data Access, and Data Management all mixed together

- Tight coupling
- Hard to maintain
- Difficult to test
- Hard to scale
- Changes impact multiple areas
- Low reusability

BENEFITS OF SEPARATION OF CONCERNS

MAINTAINABILITY
Changes in one layer have minimal impact on others.

REUSABILITY
Components can be reused across projects.

TESTABILITY
Each layer can be tested in isolation.

SCALABILITY
Layers can be scaled independently.

SECURITY
Security rules can be applied at appropriate layers.

FLEXIBILITY
Easier to adapt and extend the system.

TRY IT YOURSELF — EXERCISE 1: LAYER DIAGRAM

Reinforces: the presentation / service / repository layering you just saw, applied to one real request.

STEP	WHAT TO DO
Goal	Sketch API adapter → CustomerService → CustomerRepository for Northstar's activate(CUS-1002)
Boxes	Draw three boxes: API adapter, CustomerService, CustomerRepository
Arrows	Label activate(CUS-1002) flowing inward; the updated Customer returned outward
Correlation	Note that lab-request-001 crosses the API edge and is logged again in the service
Deliverable	Three-layer diagram saved to notes/lab15-layers.md — pre-lab only, no code yet
Reference	exercises/exercise-01-layer-diagram.md

```
[API adapter]  --activate(CUS-1002)-->  [CustomerService]
        ^                                        |
        |                                  --activate(id)-->
   Customer returned                      [CustomerRepository]

// correlation lab-request-001 crosses the API edge, logged again in the service
```

TRY IT NOW Complete Exercise 1 before continuing — see exercises/exercise-01-layer-diagram.md.

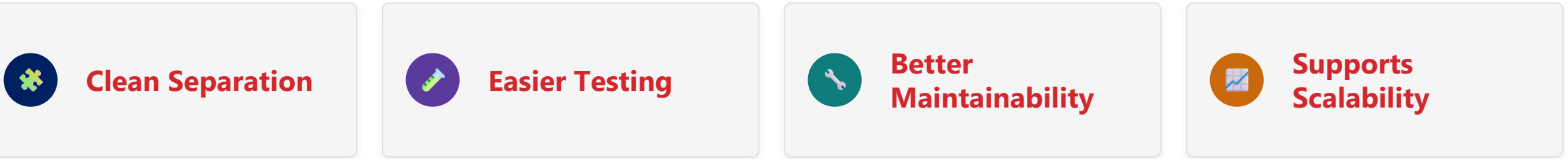
SERVICE LAYER PATTERN

Encapsulates business logic in a dedicated layer between the controller and the repository layer.

SERVICE RESPONSIBILITIES (WHO DOES WHAT)

- **Application service —** coordinates the use case, defines the transaction boundary, calls repositories/collaborators, and translates failures.
- **Domain object or domain service —** implements business rules, protects invariants, and performs domain calculations.
- **Validator or policy —** handles reusable rule evaluation where appropriate (e.g., input shape, cross-field checks).

BENEFITS



Example flow: Controller → Service (validate, apply rules) → Repository (save) → Service → Controller responds.

KEY TAKEAWAY The Service Layer Pattern keeps business logic organized, reusable, and independent of the web and data access layers.

SERVICE LAYER PATTERN — CODE EXAMPLE

A concrete, constructor-injected implementation of the CustomerService interface, ready for Lab 15.

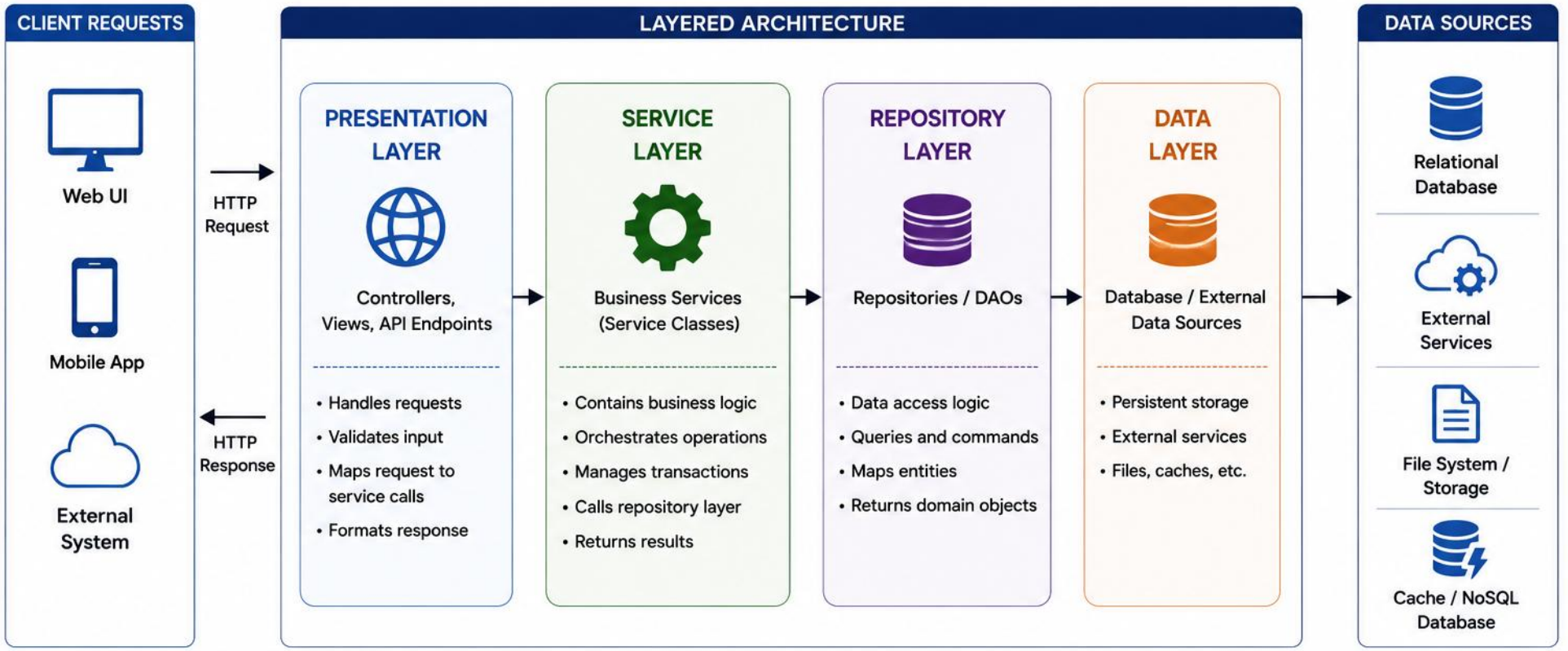
PLAIN JAVA EXAMPLE (LAB)

```
public final class DefaultCustomerService
        implements CustomerService {
    private final CustomerRepository repository;
    private final CustomerValidator validator;
    public DefaultCustomerService(
            CustomerRepository repository,
            CustomerValidator validator) {
        this.repository = repository;
        this.validator = validator;
    }
}
```

In Spring, the implementation may later be registered with @Service, and transaction boundaries may use @Transactional.

KEY TAKEAWAY The Service Layer Pattern keeps business logic organized, reusable, and independent of the web and data access layers.

SERVICE LAYER PATTERN



BENEFITS OF SERVICE LAYER PATTERN

SEPARATION OF CONCERNS
Keeps business logic separate from presentation and data access logic.

REUSABILITY
Business services can be reused across multiple controllers and clients.

TESTABILITY
Business logic can be tested independently from other layers.

MAINTAINABILITY
Changes in one layer do not impact other layers.

SCALABILITY
Supports scaling by isolating business logic.

TRANSACTION MANAGEMENT
Centralized transaction handling in the service layer.

REPOSITORY LAYER PATTERN

The Repository abstracts data access — the service layer works with domain objects, not SQL or persistence details.

KEY RESPONSIBILITIES

- Encapsulate data access logic (CRUD operations).
- Hide persistence details (JPA, JDBC, etc.) from the service layer.
- Provide a collection-like interface for domain objects.
- Enable easy mocking for unit tests.

SPRING DATA JPA EXAMPLE

```
public interface UserRepository
  extends JpaRepository<User, Long> {
  Optional<User> findByEmail(String email);
}
```

KEY TAKEAWAY The Repository Layer isolates persistence concerns, so your service layer can evolve independently of the database.

REPOSITORY LAYER PATTERN — BEST PRACTICES

Where repository abstractions help, and where controllers should not reach past the service.

BEST PRACTICES

- Repositories encapsulate persistence operations and domain-oriented queries. They should not own orchestration, workflow decisions, or business-policy evaluation. Queries like findActiveCustomersByRegion(...) or existsByEmail(...) belong here — workflow does not.
- In the standard layered architecture used in this course, controllers call application services rather than repositories directly. Specialized read models may use separate query handlers in more advanced designs.
- Introduce repository abstractions where they protect domain boundaries, support testing, or isolate persistence choices — avoid wrappers that merely duplicate a framework interface. This module's custom CustomerRepository interface is justified: it separates the service from the Map-backed implementation.

Repository flow: Service calls repo method → JPA executes query → maps results to entities → returns to service.

KEY TAKEAWAY The Repository Layer isolates persistence concerns, so your service layer can evolve independently of the database.

REPOSITORY LAYER PATTERN — CODE EXAMPLE

A repository that only persists — the transition rule stays in the service, exactly as the best practices require.

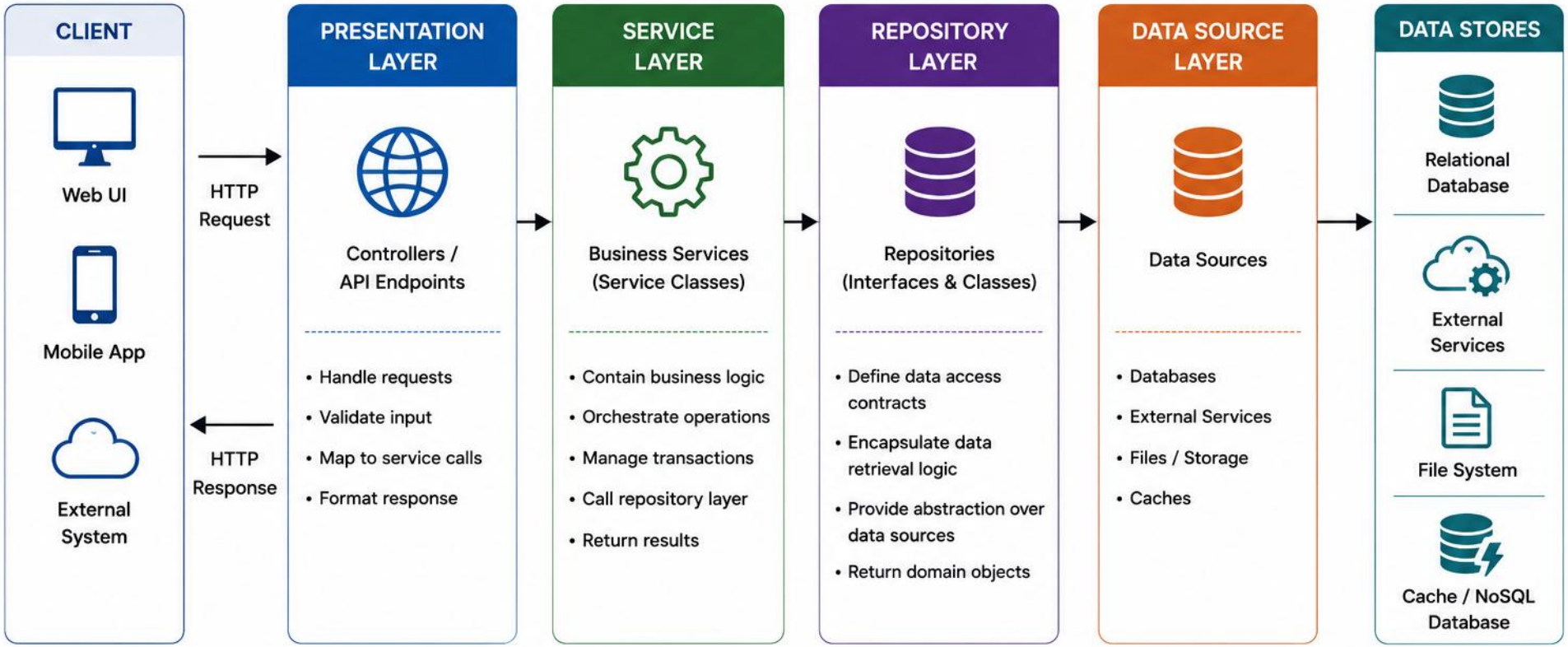
PLAIN JAVA EXAMPLE — WHO OWNS THE RULE

```
public class DefaultCustomerService implements CustomerService {
    private final CustomerRepository repository;

    @Override
    public Customer activate(String customerId) {
        Customer customer = repository.findById(customerId)
            .orElseThrow(() -> new CustomerNotFoundException(customerId));

        customer.activate();              // business rule: repo never decides this
        repository.save(customer);
        return customer;
    }
}
```

The CustomerRepository interface exposes only findById() and save() — no activateCustomer() method to hide a rule inside.

KEY TAKEAWAY Keep repositories dumb — CRUD and persistence mapping only. Business rules like activate() belong in the service, never the repository.

REPOSITORY LAYER PATTERN



KEY BENEFITS

ABSTRACTION
Hides data access details from business logic.

SEPARATION OF CONCERNS
Business logic is separate from data access logic.

TESTABILITY
Repositories can be mocked for unit testing.

FLEXIBILITY
Easy to switch data sources without changing business logic.

MAINTAINABILITY
Centralized data access logic reduces code duplication.

TRY IT YOURSELF — EXERCISE 2: REPOSITORY BOUNDARY

Reinforces: the repository best practices you just saw — persistence only, never business rules.

STEP	WHAT TO DO
Goal	List what belongs in the repository versus what belongs in the service
Repo owns	CRUD by id, existence checks, persistence mapping
Service owns	The transition matrix, notifier calls, domain exceptions
Anti-pattern	repo.activateCustomer(id) — a repository method that hides a business rule
Deliverable	Crisp ownership list saved to notes/lab15-repo-boundary.md
Reference	exercises/exercise-02-repo-boundary.md

```
// ANTI-PATTERN -- do not do this
repo.activateCustomer(id);   // business rule buried inside the repository

// CORRECT -- repository stays dumb, service owns the rule
Customer c = repo.findById(id);
service.applyTransition(c, ACTIVE);  repo.save(c);
```

TRY IT NOW Complete Exercise 2 before continuing — see exercises/exercise-02-repo-boundary.md.

BUSINESS RULE PLACEMENT

Business rules encode the 'why' of your application — where a rule belongs matters as much as how it's written.

RULE TYPE	APPROPRIATE LOCATION
Input shape	DTO validation
Entity invariant	Domain object
Multi-entity rule	Domain / application service
Workflow orchestration	Application service
Database integrity	Repository / database constraint
Authorization policy	Application / security policy

- **Name rules for what they enforce —** ensureCustomerCanBeActivated(), calculateEligibleDiscount(), rejectDuplicateEmail(), verifyCreditLimit(), applyDiscount().
- **Separate simple & complex rules —** simple validation vs. complex domain logic.
- **Avoid duplication —** extract shared rules into reusable methods/classes.
- **Make rules testable —** pure functions where possible; easy to unit test.
- **Document business intent —** comment the why, not the how.

Domain-object style: `customer.activate();` *— preferable when the entity itself owns the invariant.*

Procedural service-only style:
```
customerService.validateStatusTransition(customer.getStatus(), ACTIVE);
customer.setStatus(ACTIVE);
```

Common types of business rules: Validation, Calculation, Authorization, Workflow, and Integrity rules.

KEY TAKEAWAY Well-managed business rules make your application easier to understand, test, and evolve as requirements change.

BUSINESS RULE PLACEMENT — CODE EXAMPLE

Two ways to enforce the same invariant — a service that checks the rule, or an entity that owns it.

PROCEDURAL STYLE (SERVICE-ONLY)

```
public class CustomerService {
    public void activate(Customer c) {
        if (c.getStatus() != PROSPECT) {
            throw new IllegalStateException(
                "cannot activate");
        }
        c.setStatus(ACTIVE);
    }
}
// nothing stops other code from
//     calling setStatus() directly
```

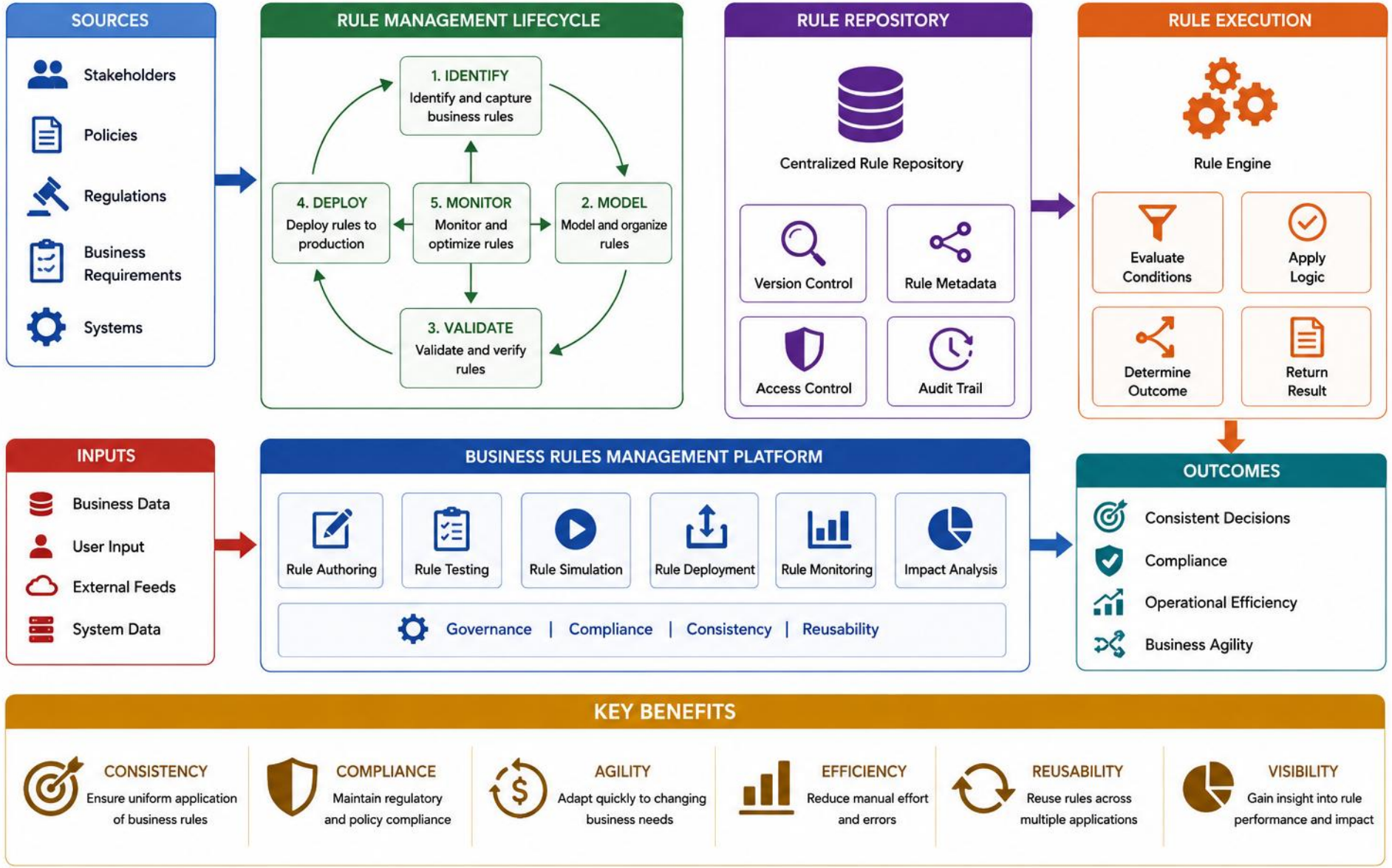
DOMAIN-OBJECT STYLE (PREFERRED)

```
public class Customer {
    private Status status;

    public void activate() {
        if (status != Status.PROSPECT) {
            throw new IllegalStateException(
                "cannot activate");
        }
        status = Status.ACTIVE;
    }
}
```

Both versions compile. Only the domain-object style makes it impossible to bypass activate() and set status directly.

KEY TAKEAWAY Prefer the domain-object style when the entity owns its own invariant — it makes illegal state changes impossible, not just discouraged.

BUSINESS RULES MANAGEMENT



TRY IT YOURSELF — EXERCISE 3: TRANSITION MATRIX

Reinforces: business rule placement — where the customer status-transition rule actually lives.

STEP	WHAT TO DO
Goal	Tabulate the allowed and forbidden Northstar customer status transitions
Copy matrix	Recreate the reference table; decide the ACTIVE→ACTIVE policy in one word
Amina check	CUS-1001 is already ACTIVE — activate should be rejected or a no-op per your policy
Illegal list	Write down two illegal transitions you will throw an exception on later
Boundary	Mark explicitly: HTTP status-code mapping for exceptions waits for Lab 16
Reference	exercises/exercise-03-transition-matrix.md

```
From        To         Allowed?
PROSPECT -> ACTIVE     yes   // Ravi activate
ACTIVE   -> ACTIVE     no-op or reject -- decide
ACTIVE   -> PROSPECT   no
```

TRY IT NOW Complete Exercise 3 before continuing — see exercises/exercise-03-transition-matrix.md.

DEPENDENCY INJECTION REVIEW

DI keeps the service layer testable — dependencies stay visible, explicit, and easy to swap for mocks or fakes.

EXAMPLE: CONSTRUCTOR INJECTION

```
@Service
public class OrderService {
  private final OrderRepository repo;
  public OrderService(OrderRepository repo) {
    this.repo = repo;
  }
}
```

Use constructor injection for required collaborators. Setter injection may be appropriate for optional or reconfigurable collaborators. Avoid field injection because it hides dependencies and complicates testing.

WHY IT MAKES SERVICES TESTABLE

- Dependencies are visible in the constructor signature — no hidden wiring to trace.
- Fields are final/immutable references — a wired service can't be reconfigured behind your back.
- Easy manual wiring — plain `new DefaultCustomerService(repo, validator)` works without a framework.
- Easy to mock or fake — pass a test double directly into the constructor.
- No hidden service locator — collaborators aren't pulled from a static registry mid-method.
- Avoid unnecessary optional constructor dependencies — they signal an unclear responsibility.

KEY TAKEAWAY Constructor injection makes your services loosely coupled, explicit about dependencies, and easy to test in isolation.

DEPENDENCY INJECTION REVIEW

WHAT IS DEPENDENCY INJECTION?

A design pattern where an object receives its dependencies from an external source rather than creating them itself.

HOW IT WORKS

1. CLIENT — Requests Service
2. DI CONTAINER — Creates and manages objects and dependencies
3. SERVICE — Gets dependencies injected
4. DEPENDENCIES — Ready to use

TYPES OF INJECTION

Constructor Injection
Dependencies are provided via constructor.

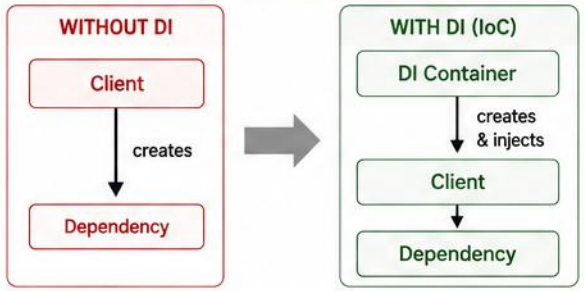
Setter Injection
Dependencies are provided via setter methods.

Field Injection
Dependencies are injected directly into fields.

DEPENDENCY INJECTION CONTAINER

1. **Configuration** — Define components and their dependencies
2. **Container Initialization** — Container reads configuration and creates objects
3. **Dependency Resolution** — Container resolves dependencies and wires objects together
4. **Object Delivery** — Fully initialized objects are provided to clients

INVERSION OF CONTROL (IoC)



KEY BENEFITS

- **Loose Coupling** — Reduces dependencies between classes.
- **Testability** — Easy to mock and unit test.
- **Maintainability** — Changes in one component have minimal impact.
- **Scalability** — Easy to extend and manage complex systems.
- **Reusability** — Components can be reused across applications.

EXAMPLE

```
class OrderService {
  private PaymentService
    paymentService;

  // Constructor Injection
  OrderService(PaymentService
    paymentService) {
    this.paymentService =
      paymentService;
  }
}
```

DI CONTAINER
Creates PaymentService and injects into OrderService

```
class PaymentService {
  // Implementation
}
```

TRY IT YOURSELF — EXERCISE 4: INTERFACE & CTOR SKETCH

Reinforces: constructor injection — sketch CustomerService's methods and dependencies on paper.

STEP	WHAT TO DO
Goal	Sketch CustomerService's methods and constructor dependencies before the timed lab
Interface	Methods: findById(customerId), activate(customerId)
Constructor	Deps: CustomerRepository, optional CustomerNotifier — JDK-style ctor injection sketch
No framework magic	Prefer an explicit constructor over field injection, per this module's standards
Deliverable	Interface + constructor sketch ready to type in during Lab 15
Reference	exercises/exercise-04-interface-ctor-sketch.md

```
public interface CustomerService {
    Customer findById(String customerId);
    Customer activate(String customerId);
}

public DefaultCustomerService(CustomerRepository repository, CustomerNotifier notifier)
```

TRY IT NOW Complete Exercise 4 before continuing — see exercises/exercise-04-interface-ctor-sketch.md.

TRANSACTION BOUNDARY CONCEPT

A preview of transaction boundaries — one use case is one local unit of work. Detailed propagation and isolation rules are covered in the dedicated transaction module.

THE CORE IDEA

- One use case = one logical unit of work.
- Local database changes should succeed or fail together.
- Application services commonly define the boundary.
- Detailed propagation and isolation rules are covered in the dedicated transaction module.

EXAMPLE (SPRING) — LOCAL OPERATIONS ONLY

```
@Transactional
public void placeOrder(OrderRequest request) {
    orderRepository.save(order);

inventoryRepository.reserveStock(request.items());
}
```

REMOTE CALLS ARE NOT PART OF THE TRANSACTION

- Remote calls require distributed-workflow patterns such as outbox, saga, compensation, or idempotent retries — not a normal local database transaction.
- Keep the transaction short and scoped to one use case; do not hold it open across a network call.

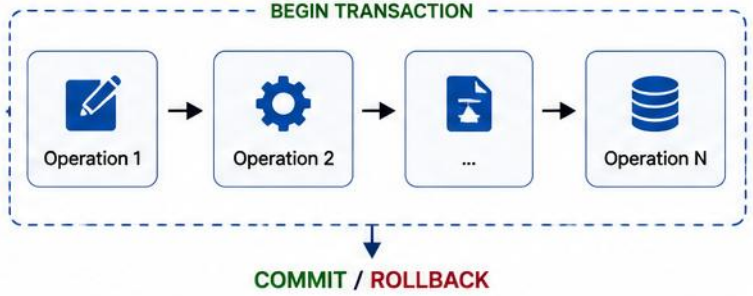
Transaction rollback behavior depends on the framework and exception configuration. In Spring, unchecked exceptions normally trigger rollback by default; checked exceptions may require explicit configuration.

KEY TAKEAWAY A transaction boundary is a local unit of work — commonly scoped to one application-service use case — and kept free of remote calls.

TRANSACTION BOUNDARIES

WHAT IS A TRANSACTION BOUNDARY?

A transaction boundary defines the scope of work that must be completed as a single unit.
Either all operations succeed (COMMIT) or all fail (ROLLBACK).



ACID PROPERTIES

- **A — Atomicity**: All operations succeed or none.
- **C — Consistency**: Database remains in a valid state.
- **I — Isolation**: Concurrent transactions do not interfere.
- **D — Durability**: Committed changes are permanent.

WHERE TO DEFINE TRANSACTION BOUNDARIES?

PRESENTATION LAYER

UI / API Controller

Should be thin.
Not the right place for transactions.

SERVICE LAYER (RECOMMENDED)

Define transaction boundaries around business use cases.

Example Use Case:
Transfer Money
(debit from A, credit to B)

REPOSITORY LAYER

Execute data operations.
Should not control transaction boundaries.

DATABASE

Enforces ACID at the database level.

EXAMPLE: TRANSFER MONEY

1. BEGIN TRANSACTION
2. Read balance of Account A
3. Debit amount from Account A
4. Credit amount to Account B
5. Update transaction records
6. COMMIT

All succeed → COMMIT

Any failure → ROLLBACK

GOOD PRACTICES

Align with Business Use Cases
One transaction per business operation.

Keep it Short
Minimize duration to reduce locks and improve performance.

Do Not Cross Boundaries Unnecessarily
Avoid transactions spanning external systems.

Handle Exceptions Properly
Rollback on failure, commit on success.

Monitor and Log
Track slow or failed transactions.

BAD EXAMPLES (AVOID)

Long-Running Transactions
Holding locks for too long impacts concurrency.

User Interaction Inside Transaction
Never wait for user input inside a transaction.

External Calls Inside Transaction
APIs, messaging, or I/O calls can cause timeouts and inconsistencies.

SERVICE COMMUNICATION — ARCHITECTURE PREVIEW

This module focuses on in-process service-layer design. Cross-service and distributed communication are previewed here only; a dedicated module covers them in depth.

STYLE	BEST FIT	MAIN TRADE-OFF
In-process call	Same application or module	Tight runtime coupling
REST / HTTP	Simple, interoperable request-response	Temporal coupling
gRPC	Typed, high-throughput internal calls	Stronger tooling / protocol coupling
Messaging / events	Asynchronous workflows and decoupling	Eventual consistency and complexity

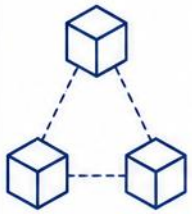
This module's service layer (and Lab 15) uses only in-process, direct method calls — CustomerService calls CustomerRepository directly, with no network hop.

Messaging supports asynchronous decoupling and load buffering, but requires explicit handling of retries, duplicates, ordering, and eventual consistency.

KEY TAKEAWAY Choose a communication style deliberately — most of this module's work stays in-process; cross-service styles get full treatment in the distributed-systems module.

SERVICE-TO-SERVICE COMMUNICATION

OVERVIEW

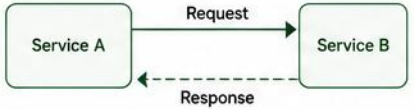


Service-to-Service communication enables microservices to exchange data and coordinate actions across the system.

COMMUNICATION PATTERNS

SYNCHRONOUS

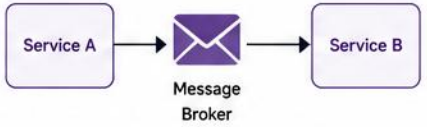
Real-time request / response



Use Cases:
- Real-time data retrieval
- Immediate validation
- Short-running operations

ASYNCHRONOUS

Event-driven messaging



Use Cases:
- Decoupled processing
- Long-running tasks
- Event notifications

COMMON PROTOCOLS

REST / HTTP
JSON over HTTP/HTTPS
Simple, widely used

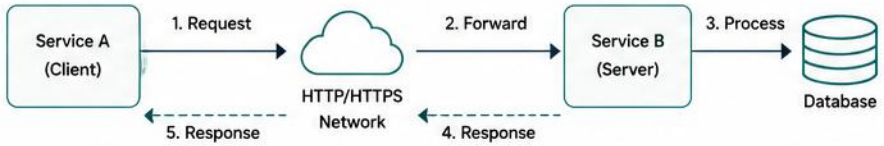
gRPC
HTTP/2 + Protocol Buffers
High performance

MESSAGING (AMQP/Kafka)
Asynchronous, event-driven
High reliability, scalable

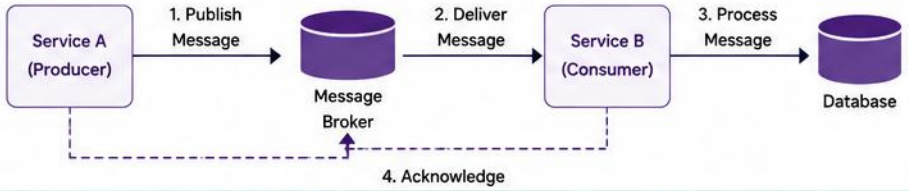
SOAP
XML-based protocol
Enterprise integration

COMMUNICATION FLOWS

SYNCHRONOUS FLOW (REST EXAMPLE)



ASYNCHRONOUS FLOW (MESSAGE QUEUE EXAMPLE)



BEST PRACTICES

- Use appropriate pattern based on use case
- Implement timeouts and retries
- Use circuit breaker for fault tolerance
- Secure communication (TLS, authN/Z)
- Log and monitor all communications
- Use API versioning for compatibility

DESIGN CONSIDERATIONS

DEPENDENCY MANAGEMENT	Minimize inter-service dependencies to reduce coupling.
PERFORMANCE	Choose protocols and patterns that meet latency and throughput needs.
RELIABILITY	Handle failures gracefully and ensure message delivery.
SCALABILITY	Design for horizontal scaling and high availability.
OBSERVABILITY	Implement tracing, metrics, and distributed logging.

CHALLENGES

NETWORK FAILURES	Handling timeouts, connection issues, and partitions.
PARTIAL FAILURES	Some services succeed while others fail.
DATA CONSISTENCY	Maintaining consistency across multiple services.
LATENCY	Network calls add latency and affect performance.
SECURITY	Protecting data in transit and validating service identities.

SERVICE DESIGN QUALITY ATTRIBUTES

Six quality attributes to check for any service — on top of the architecture patterns already covered in this module.

Correctness

Each use case behaves as specified, with rules enforced deliberately.

Testability

Program to interfaces so behavior can be verified in isolation.

Security

Validate inputs and enforce authorization at every service boundary.

Observability

Logging, metrics, and tracing built in.

Performance

Avoid N+1 queries; use caching where appropriate.

Reliability

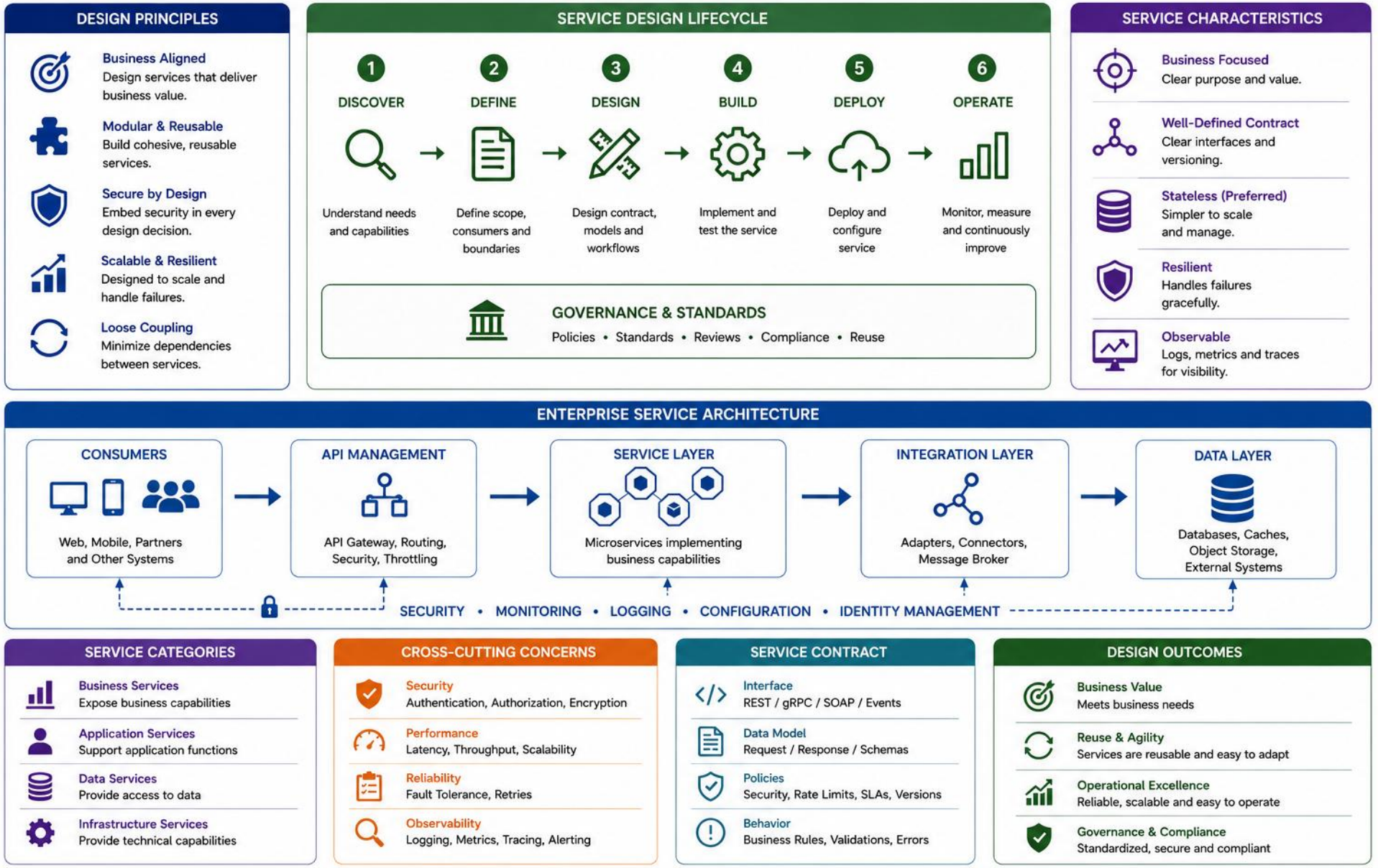
Persist durable state externally; avoid request-specific mutable state in instances.

Data ownership — *In microservice architectures, each independently deployed service should generally own its data. In a modular monolith, enforce clear ownership boundaries even when infrastructure is shared.*

Statelessness — *Keep service instances free of request-specific mutable state where possible. Persist durable state externally so multiple instances can process requests safely.*

KEY TAKEAWAY Use these attributes as a review checklist for any service you design — not a new set of architecture rules on top of what you've already learned.

ENTERPRISE SERVICE DESIGN



COMMON MISTAKES & PREFERRED DESIGN (1 OF 2)

Each mistake paired with its impact and the preferred design that avoids it — part 1 of 2.

MISTAKE	IMPACT	PREFERRED DESIGN
Business logic in controllers	Tight coupling, poor testability	Keep business logic in the service — not in controllers or repositories.
Bypassing the service layer (controllers call repositories directly)	Violates separation of concerns	Follow layered architecture: Controller → Service → Repository, always.
Anemic service layer	Thin services, duplicated logic	A purely pass-through service may be unnecessary if it adds no boundary, policy, orchestration, or abstraction. A thin application service can still be valid when domain objects contain the business behavior.
Incorrect transaction boundaries	Performance issues, inconsistent data	Scope one local unit of work per use case, defined at the application service.

KEY TAKEAWAY Good design is about making the right things easy and the wrong things hard — use the preferred-design column as your default.

ANEMIC VS. RICH DOMAIN MODEL — CODE EXAMPLE

The 'anemic service layer' mistake from the table above, made concrete — logic that leaks out of the object it belongs to.

ANEMIC — LOGIC LEAKS INTO THE SERVICE

```
public class Customer {
    private String status;
    public String getStatus() {
        return status;
    }
    public void setStatus(String s) {
        status = s;
    }
}
public class CustomerService {
    public void activate(Customer c) {
        c.setStatus("ACTIVE");  // rule here
    }
}
```

RICH — THE OBJECT PROTECTS ITS OWN STATE

```
public class Customer {
    private Status status;

    public void activate() {
        if (status != Status.PROSPECT) {
            throw new IllegalStateException();
        }
        status = Status.ACTIVE;
    }
}
// service now just calls customer.activate()
```

Both classes compile. The anemic version scatters Customer's rules across every service that touches it; the rich version keeps them in one place.

KEY TAKEAWAY An anemic domain model pushes all behavior into services, leaving entities as dumb data bags — keep invariant-protecting logic on the object that owns the state.

COMMON MISTAKES & PREFERRED DESIGN (2 OF 2)

Each mistake paired with its impact and the preferred design that avoids it — part 2 of 2.

MISTAKE	IMPACT	PREFERRED DESIGN
Tight coupling between services	Hard to change, test, deploy independently	Use loose coupling — interfaces between services, constructor injection.
Ignoring error handling & logging	Hard to troubleshoot and monitor in production	Handle errors where meaningful recovery or translation is possible. Log once at an appropriate boundary with correlation and context; avoid duplicate stack traces across layers.
Weak or missing unit tests for business logic	Regressions ship silently; refactors become risky	Write unit tests — test business logic in isolation with mocks.

KEY TAKEAWAY Good design is about making the right things easy and the wrong things hard — use the preferred-design column as your default.

UNIT TESTING THE SERVICE LAYER — CODE EXAMPLE

Business logic tested in isolation with a fake repository — no database, no Spring context, no framework required.

JUNIT 5 EXAMPLE — FAKE REPOSITORY, NO MOCKING FRAMEWORK

```
class DefaultCustomerServiceTest {

    @Test
    void activateTransitionsProspectToActive() {
        CustomerRepository fakeRepo = new InMemoryCustomerRepository();
        Customer prospect = new Customer("CUS-1002", Status.PROSPECT);
        fakeRepo.save(prospect);
        CustomerService service =
            new DefaultCustomerService(fakeRepo, new CustomerValidator());

        Customer activated = service.activate("CUS-1002");

        assertEquals(Status.ACTIVE, activated.getStatus());
    }
}
```

KEY TAKEAWAY Program to the CustomerService and CustomerRepository interfaces so business logic can be unit-tested with a fake or mock — no database or framework required.

COMMON DESIGN MISTAKES

1

High Coupling

Components depend on too many others, making changes risky and difficult.

2

Low Cohesion

A component has too many responsibilities, violating the single responsibility principle.

3

Code Duplication

Duplicated logic increases maintenance effort and the chance of inconsistencies.

4

Leaky Abstractions

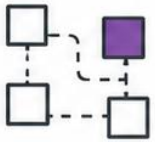
Internal implementation details are exposed, causing tight coupling and fragility.

5

Ignoring SOLID Principles

Violating SOLID principles leads to unmaintainable and inflexible designs.

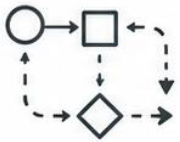
6



God Object / God Class

A single class does too much, becomes complex and hard to test.

7



Poor Layering

Mixing concerns across layers leads to spaghetti dependencies.

8

Ignoring Data Boundaries

Sharing data directly across services breaks encapsulation and causes side effects.

9

Over-Engineering

Building overly complex solutions for simple problems wastes time and resources.

10

No Extensibility (Closed Design)

Designs that don't anticipate change are hard to extend and adapt.

TRY IT YOURSELF — EXERCISE 5: ACTIVATE RAVI (STARTER)

Reinforces: everything so far — a fill-in-the-blank starter, not a from-scratch build.

STEP	WHAT TO DO
Goal	Complete the fill-in-the-blank pseudocode for activate(CUS-1002)
Starter	TODO / fill-in-the-blank skeleton — replace every _____; blanks do not run as-is
Fixtures	Ravi starts PROSPECT and must end ACTIVE; Amina (CUS-1001) is already ACTIVE
Repo boundary	Write: repository saves state; it does not decide PROSPECT→ACTIVE
Common mistake	Putting the transition if-checks inside the repository instead of the service
Reference	exercises/exercise-05-fill-activate-ravi-todos.md

```
customer = repo.findById(_____)
if customer is null -> throw _____
if status is not _____ -> throw _____
set status to _____
repo._____(customer)
log correlation _____
```

TRY IT NOW Complete Exercise 5 (fill in every blank) before continuing — see exercises/exercise-05-fill-activate-ravi-todos.md.

TRY IT YOURSELF — EXERCISE 6: LAB 15 PREP CHECKLIST

Reinforces: confirm every pre-lab artifact exists before starting the graded Lab 15 build.

STEP	WHAT TO DO
Goal	Confirm service-layer prep is complete — without starting Lab 15 itself
Artifacts	Confirm the diagram, matrix, interface sketch, and activate TODOs all exist
Fixtures	Recall Ravi PROSPECT→ACTIVE (happy path) and Amina already ACTIVE (rejected path)
Boundary	Write: pre-lab only — prepare for the lab, do not complete full Lab 15 now
Pass/Fail	Pass if the repository-boundary sentence exists; else revisit Exercise 5
Reference	exercises/exercise-06-lab15-prep-checklist.md

```
[ ] lab15-layers.md          // Exercise 1 -- layer diagram
[ ] lab15-repo-boundary.md   // Exercise 2 -- repo vs service
[ ] transition matrix        // Exercise 3 -- PROSPECT/ACTIVE rules
[ ] interface + ctor sketch  // Exercise 4 -- CustomerService
[ ] lab15-activate-todos.md  // Exercise 5 -- filled, no blanks left
```

TRY IT NOW Complete Exercise 6 before Lab 15 — see exercises/exercise-06-lab15-prep-checklist.md.

LAB OVERVIEW – SERVICE LAYER DESIGN

Design and implement a clean, layered service architecture for the Northstar CRM Customer Management Platform — no Spring required yet.

WHAT YOU WILL BUILD

- A CustomerService layer for the Northstar CRM Customer Management Platform.
- Add customers and change status: PROSPECT → ACTIVE, with illegal transitions rejected.
- Validate identity, email uniqueness, and status transitions before saving.
- Status-transition rules may live in a validator or in domain behavior — both are valid.

TECH STACK

ITEM	VALUE
`Language`	Java 21
`Framework`	Plain Java (manual DI)
`Data Access`	Repository interface (Map-backed)
`Database`	In-memory Map (no DB)

ARCHITECTURE FLOW

Facade **CustomerService** **CustomerRepository**

CustomerValidator (or a domain policy) is attached to CustomerService and is called before persisting changes.

Transition rule: the lab branches from Lab 14's facade, and adds failure experiments and evidence capture after testing.

KEY TAKEAWAY You will build a robust, enterprise-grade service layer that is maintainable, testable, and aligned with best practices.

LAB TASKS AND DELIVERABLES

Build the repository, service, and validator layers, then activate and reject customer status transitions.

#	TASK	DETAILS	FORMAT
1	Project Setup	Copy lab14-crm to lab15-crm; add the repository package.	n/a
2	Repository Layer	CustomerRepository interface + InMemoryCustomerRepository (private Map).	.java
3	Service Layer	CustomerService interface + DefaultCustomerService with constructor DI.	.java
4	Validator	CustomerValidator: identity, email uniqueness, status transitions.	.java
5	Status Transitions	Activate CUS-1002 PROSPECT→ACTIVE; reject ACTIVE→PROSPECT — the domain exception carries a stable error code, and the facade adds the correlation ID.	.java
6	Testing & Docs	CustomerValidatorTest + DefaultCustomerServiceTest (JUnit 5): create, duplicate email, valid/invalid transition, repo-not-called-on-failure, error-code checks; transition table in README.	Test suite

KEY TAKEAWAY Focus on clean design, separation of concerns, and maintainability throughout the lab.

LAB 15 — SUCCESS CRITERIA

How to know your service and repository layers are done right.

SUCCESS CRITERIA

- No Map/SQL leaks past the repository; illegal transitions rejected with a stable error code, and the facade attaches the correlation ID.
- The repository enforces email uniqueness as the final integrity guarantee — service validation only improves the error message.
- Service depends on the CustomerRepository interface (not InMemoryCustomerRepository); constructor args are non-null; returned collections are not mutable internal references; no static/global repository state.
- CustomerValidatorTest and DefaultCustomerServiceTest both pass and run independently, using a mocked or in-memory fake repository.

KEY TAKEAWAY Focus on clean design, separation of concerns, and maintainability throughout the lab.

SERVICE LAYER: DEFINITION OF DONE

A checklist to confirm your service-layer design is complete before calling a use case done.

1. The use case has one clear entry point.
2. The controller or facade contains no business workflow.
3. Business invariants are placed deliberately.
4. Dependencies are explicit and constructor-injected.
5. Persistence is accessed through an abstraction.
6. Transaction scope matches one local unit of work.
7. Remote calls are not assumed to be part of a local transaction.
8. Errors have stable codes and useful context.
9. Logging is not duplicated.
10. Unit tests cover successful and rejected workflows.
11. Infrastructure can change without rewriting business logic.

Use this as a pre-review checklist for any new use case in the service layer.

KEY TAKEAWAY A service layer that satisfies this checklist is maintainable, testable, and safe to extend.

KNOWLEDGE CHECK (1 OF 4)

Test your understanding of business logic and service layer design.

1. A controller validates eligibility, checks duplicate email, saves via a repository, and sends a notification. What should be refactored?

A. Nothing — this is standard controller responsibility

B. Move orchestration into a service; keep controllers thin

C. Move the repository call into the controller for speed

D. Add more validation directly in the repository

2. Where are transaction boundaries most commonly defined in a layered application?

A. In the Controller

B. In the Repository

C. Around application-service use cases

D. In the Database

KEY TAKEAWAY The Service Layer owns business logic and transactions — keep it rich, testable, and well-bounded.

KNOWLEDGE CHECK (2 OF 4)

Two more questions on business logic and service layer design.

3. Which is a sign of an anemic service layer?

A. Rich business logic in services

B. Services only delegate to repositories

C. Well-tested business rules

D. Clear separation of concerns

4. Why prefer constructor injection?

A. It's faster at runtime

B. Dependencies are explicit and easy to test

C. It requires less code

D. It avoids using interfaces

KEY TAKEAWAY The Service Layer owns business logic and transactions — keep it rich, testable, and well-bounded.

KNOWLEDGE CHECK (3 OF 4)

Two questions on mistakes, transactions, repositories, and coupling.

5. What's a common mistake in layered design?

A. Using interfaces between layers

B. Controllers calling repositories directly

C. Writing unit tests for services

D. Using dependency injection

6. A service writes to the database, then calls a remote payment API inside @Transactional. What is the main problem?

A. The code won't compile

B. The local transaction can't atomically roll back the remote call

C. @Transactional doesn't work with repositories

D. The service should use more repositories

KEY TAKEAWAY Good design is about making the right things easy and the wrong things hard.

KNOWLEDGE CHECK (4 OF 4)

Final two questions, plus the full answer key.

7. What does the Repository layer do?

A. Contains business rules

B. Handles HTTP responses

C. Encapsulates data access logic

D. Manages UI state

8. Why avoid tight coupling between services?

A. It improves performance

B. It makes services hard to change and test

C. It's required by Spring Boot

D. It reduces the number of classes

ANSWER KEY

- **1-B 2-C 3-B 4-B 5-B 6-B 7-C 8-B**

KEY TAKEAWAY Good design is about making the right things easy and the wrong things hard.

"Good design makes the right things easy and the wrong things hard."

MODULE 15 COMPLETE

Business Logic and Service Layer Design

You can now design clean, testable service and repository layers with proper transaction management.